Almacenamiento de documentos XML y JSON.



1 Introducción 2

2 Sistemas de almacenamiento 2

2.1 Almacenamiento en sistemas de archivos 2

2.2 Bases de datos de documentos (NoSQL) 3

2.3 Bases de datos relacionales (SQL) 3

2.4 Sistemas de almacenamiento en la nube 4

2.5 Almacenamiento en cachés y sistemas en memoria 5

3 Factores para elegir la mejor opción 5

4 Procesamiento y consulta de documentos JSON. 7

5 Ejemplos de uso de algunas tecnologías 8

5.1 JQ 8

5.2 JSONiq 10

1 Introducción

En el mundo del desarrollo de software y la gestión de datos, los formatos ***XML (Extensible Markup Language)*** y ***JSON (JavaScript Object Notation)*** juegan un papel crucial en la representación e intercambio de información. Ambos formatos son ampliamente utilizados en diversas aplicaciones, desde configuraciones de software hasta servicios web y almacenamiento de datos estructurados.

Dado que XML y JSON son formatos ampliamente utilizados para el intercambio y almacenamiento de datos, elegir una estrategia adecuada para su persistencia es fundamental para garantizar:

- **Eficiencia en el acceso a los datos.**
- **Escalabilidad y rendimiento en sistemas con grandes volúmenes de información.**
- **Compatibilidad con diferentes aplicaciones y servicios.**
- **Facilidad de consulta y manipulación de la información almacenada.**

Los casos de uso comunes de ambas tecnologías en diferentes contextos se pueden resumir en la siguiente tabla:

Caso de uso	XML	JSON
Intercambio de datos en API	Menos común hoy en día, pero aún usado en algunos servicios SOAP.	Dominante en APIs REST
Almacenamiento de configuraciones	Configuración de aplicaciones en Windows y algunos sistemas embebidos.	También usado en un número importante de aplicaciones y sistemas.
Bases de datos	Usado en bases de datos orientadas a documentos (ej. ***BaseX***, ***eXist-db***). Sistemas tradicionales de bases de datos como ***MySQL***, ***Oracle***, ***DB2***, etc, también lo permiten.	Compatible con bases NoSQL como ***MongoDB***. Sistemas como ***eXist-db***, también lo permiten. Sistemas tradicionales de bases de datos como ***MySQL***, ***Oracle***, ***DB2***, etc, también lo permiten.
Documentos estructurados	Preferido en industrias que requieren validación de datos estricta, como banca y salud.	Usado en almacenamiento de datos flexibles y estructuras dinámicas.
Mensajería y protocolos de comunicación	Usado en protocolos como SOAP, XMPP y algunos servicios heredados.	Predominante en protocolos modernos como WebSockets y APIs REST.

2 Sistemas de almacenamiento

2.1 Almacenamiento en sistemas de archivos

Una de las formas más simples de almacenar documentos XML y JSON es **guardarlos como archivos en el *sistema de archivos*** del servidor o dispositivo. Los archivos pueden guardarse en formatos ***.json*** o ***.xml*** dentro de una **estructura de carpetas** organizada según la aplicación. Esta

opción es útil cuando se manejan volúmenes pequeños o medianos de datos que no requieren consultas complejas.

Ventajas:

- Fácil implementación sin necesidad de un sistema de gestión de bases de datos.
- Accesible desde cualquier lenguaje de programación con funciones estándar de lectura/escritura.
- Útil para configuraciones, logs o datos de pequeña escala.

Desventajas:

- No es eficiente para búsquedas complejas o grandes volúmenes de datos.
- Dificultad para gestionar concurrencia en entornos multiusuario.
- Falta de mecanismos de indexación y recuperación rápida.

2.2 Bases de datos de documentos (NoSQL)

Las bases de datos orientadas a documentos están diseñadas para almacenar y gestionar JSON o XML de manera eficiente. Algunos ejemplos son los siguientes:

- **MongoDB** (JSON/BSON)
- **CouchDB** (JSON)
- **BaseX** (XML). Más sencilla.
- **eXist-db** (XML y JSON). Más sencilla.

Ventajas:

- Indexación y consultas optimizadas para documentos JSON o XML.
- Escalabilidad horizontal para grandes volúmenes de datos.
- Los elementos de este tipo de base de datos son muy flexibles en cuanto al formato de cada documento, no existiendo la rigidez propia de las bases de datos relacionales en las que los registros tienen que seguir un esquema determinado.

Desventajas:

- Requiere conocimientos específicos para optimizar consultas y almacenamiento.
- Algunas bases de datos NoSQL no garantizan ACID (Atomicidad, Consistencia, Aislamiento, Durabilidad).

2.3 Bases de datos relacionales (SQL)

Los sistemas de bases de datos relacionales clásicos (***MySQL, Oracle, DB2, SQL Server, PostgreSQL,…***), han evolucionado para permitir hoy en día almacenar información en formato

XML y *JSON* y poder realizar consultas eficientes sobre ellos. Básicamente, podríamos hacerlo de dos formas distintas:

- **Una opción sencilla es almacenar los documentos *XML/JSON* en una columna de tipo TEXT o VARCHAR**. Esta opción no permite consultas eficientes dentro del contenido y tampoco ofrece indexación ni optimización para búsquedas avanzadas.

- **Usar tipos de datos específicos para JSON/XML**. De esta forma, se pueden indexar y realizar consultas más optimizadas sobre ellos y tendremos acceso al uso de funciones más avanzadas. Sin embargo, la complejidad en la estructura de las consultas crecerá y tendremos mayor consumo de espacio en comparación con bases NoSQL optimizadas para JSON/XML.

2.4 Sistemas de almacenamiento en la nube

Las soluciones en la nube ofrecen almacenamiento escalable y acceso global a los documentos JSON y XML. Ofrecen multitud de servicios relacionados, como, por ejemplo:

- Servicios de almacenamiento de objetos. Existen servicios que permiten guardar documentos XML/JSON como archivos individuales en una estructura de carpetas, de forma similar a como lo hacemos en los sistemas de archivos. Por ejemplo:
    - *AWS S3*.
    - *Google Cloud Storage*.
    - *Azure Blob Storage.*

  Ventajas:

  - Alta disponibilidad y replicación automática.
  - Se integra fácilmente con otros servicios en la nube.

  Desventajas:

  - No permite búsquedas eficientes dentro de los archivos.
  - Acceso más lento en comparación con bases de datos optimizadas.

- Bases de datos NoSQL en la nube o bases de datos relacionales. Hoy en día, la mayor parte de las bases de datos NoSQL y de las bases de datos relacionales permiten ser implementadas a través de servicios en la nube. Por ejemplo, ***MongoDB*** se puede implementar en la nube de múltiples formas. Una de ellas es hacerlo a través de la plataforma propia de la tecnología llamada ***MongoDB Atlas***. Los proveedores Cloud ofrecen servicios con una funcionalidad análoga como por ejemplo ***Amazon DynamoDB (JSON)*** o ***Azure Cosmos DB (JSON)***.

  Ventajas:

  - Alta escalabilidad y replicación automática.
  - Consultas optimizadas para estructuras JSON.

Desventajas:

- Puede ser costoso a gran escala.
- Requiere adaptación a modelos NoSQL.

2.5 Almacenamiento en cachés y sistemas en memoria

Para mejorar el rendimiento en aplicaciones con acceso frecuente a documentos JSON o XML, se pueden usar soluciones en memoria como ***Redis*** o ***Memcached***.

Su principal ventaja reside en que como garantizan que los datos **estarán únicamente en memoria** y, por lo tanto, su acceso será mucho más rápido. Sin embargo, esa ventaja conlleva una importante desventaja, que es que no son persistentes a los cambios (aunque ofrecen alternativas). **Este tipo de sistemas se utilizan normalmente como sistemas intermedios de caché, existiendo un sistema que está por encima de ellos que se utiliza como soporte estable de los datos**.

3 Factores para elegir la mejor opción

La elección del método de almacenamiento para documentos XML y JSON depende de varios factores clave. Cada solución tiene ventajas y desventajas según el volumen de datos, la estructura, la frecuencia de acceso, la complejidad de las consultas y los costos de mantenimiento. A continuación, analizamos los factores más importantes para tomar una decisión adecuada:

- **Volumen de datos.** El tamaño y la cantidad de documentos XML/JSON influyen en la elección del almacenamiento.
  - **Pequeñas cantidades de datos (MBs o pocos GBs).** Archivos en el sistema de ficheros o bases SQL con almacenamiento en columnas JSON/XML.
  - **Grandes volúmenes de datos.** En este caso, deberíamos optar por Bases de datos NoSQL orientadas a documentos, ya sean en la nube o en infraestructuras locales, o sistemas de almacenamiento de objetos en la nube como AWS S3.

- **Frecuencia de acceso y velocidad.** Dependiendo de la velocidad de acceso que necesitemos, unas soluciones serán más eficientes que otras.
  - **Para alta frecuencia de acceso y consultas rápidas necesarias.** La opción recomendada es el uso de bases de datos NoSQL.
  - **Para baja frecuencia de acceso.** La opción recomendada son el almacenamiento en sistema de ficheros convencionales o almacenamiento en sistemas de objetos en la nube.
  - **Necesidad de acceso ultrarrápido**. Para este tipo de infraestructuras en las que la velocidad de acceso resulta crítica, la mejor opción son los sistemas de caché en memoria como *Redis*.

- **Estructura de los datos**. El formato de los datos afecta la elección del almacenamiento.
    - **Si necesitamos estructuras flexibles y cambiantes**. Lo más adecuado serían las bases NoSQL como *MongoDB* o *DynamoDB*.
    - **Debemos usar datos estructurados y normalizados**. Usar bases de datos SQL con soporte para JSON/XML. En este caso, se podría incluso asegurar el formato de los contenidos con el uso de las tecnologías *XML Schema* o *JSON Schema*.

- **Escalabilidad y replicación**. Si el sistema debe manejar grandes cantidades de datos y tráfico en entornos de rápida evolución, se deben considerar opciones escalables.
    - **Escalabilidad horizontal** (añadir más servidores según crece el tráfico). Las bases de datos NoSQL están mejor adaptadas para el crecimiento del volumen de datos.
    - **Escalabilidad vertical** (mejorar el hardware del servidor). Es la que suele hacerse en los sistemas de bases SQL con soporte para JSON/XML en servidores optimizados.
    - **Alta disponibilidad y replicación automática**. Si queremos que estas opciones se gestionen de forma semiautomática, deberíamos irnos a sistemas en la nube, ya que los servicios que ofrecen los proveedores se pueden adaptar a multitud de entornos diferentes (todos ellos condicionados por el coste del servicio).

- **Coste y mantenimiento**.
    - **Bajo coste (soluciones gratuitas o autoalojadas)**. Archivos en sistemas de ficheros propios o bases de datos sencillas como *BaseX* o *eXist-db*.
    - **Coste moderado con gestión propia**. Bases de datos NoSQL o relacionales en servidores propios.
    - **Coste más alto, pero con gestión automatizada**. Servicios en la nube. Hay que valorar si lo que nos ahorramos en gastos de administración y la flexibilidad que ofrecen estos servicios nos resultan más beneficiosos.

Aunque resulta muy difícil sintetizar lo visto en los puntos anteriores de una forma esquemática, la siguiente tabla lo intenta:

Opción	Coste	Mantenimiento	Escalabilidad
Archivos en ficheros	Bajo	Bajo	Baja
SQL con JSON/XML	Medio	Medio	Escalabilidad vertical
NoSQL (MongoDB, CouchDB, …)	Medio	Medio-Alto	Escalabilidad horizontal
Bases en la nube (DynamoDB, …)	Alto	Bajo(gestionado)	Muy alta (todo lo que queramos)

4 Procesamiento y consulta de documentos JSON.

A diferencia de XML, donde ***XPath*** y ***XQuery*** dominan, en el mundo de JSON hay muchas más opciones, ya que las distintas herramientas aplican enfoques diferentes que pueden afectar a sus casos de uso. Si buscamos algo parecido a *XPath/XQuery* para JSON, ***JSONPath, JSONata y JSONiq*** son las alternativas más cercanas, pero en la práctica, **las librerías estándar** de cada lenguaje de programación son suficientes para la mayoría de los casos, si lo que queremos es trabajar directamente sobre sus ficheros de forma puntual. Si, por el contrario, estamos trabajando con bases de datos NoSQL, como, por ejemplo, MongoDB, **deberíamos usar el lenguaje propio del sistema gestor**. A día de hoy, **el mundo NoSQL carece de estándares de consultas**, como ocurre en el caso de las bases de datos relacionales con el *SQL*. Esto es así porque son tecnologías más recientes y también porque existen muchos tipos de bases de datos NoSQL (*documentales, clave-valor, grafos, columnares, de tiempo real, de memoria, …*)

Enumeremos algunas de las principales tecnologías, indicando brevemente sus características:

Tecnología	Descripción	Pros	Contras
JSONPath	Similar a *XPath* pero para JSON. Permite consultas simples con una sintaxis intuitiva.	- Sencillo de usar - Amplio soporte en muchos lenguajes	- Su estándar no se ha desarrollado totalmente. - Limitado en expresividad (no soporta transformaciones avanzadas)
JSONata	Lenguaje de consultas y transformaciones inspirado en *XPath/XQuery*.	- Más potente que *JSONPath* - Soporta expresiones y funciones avanzadas	- No tan ampliamente adoptado como *JSONPath*
JQ	Herramienta de línea de comandos para transformar JSON. Usa una sintaxis funcional.	- Muy potente para transformaciones - Ideal para scripting en shell	- Sintaxis no tan intuitiva para principiantes.
JQAL	Extiende *JQ* con capacidades adicionales. Se centra en aplicaciones más específicas.	- Más expresivo que *JQ*	- Menos conocido y con menos adopción.
JSONiq	Inspirado en *XQuery* pero para JSON. Se usa para procesar JSON en bases de datos.	- Muy potente para bases de datos JSON. - Compatible con *XQuery*	- Poca adopción fuera del mundo académico.
Librerías estándar (JavaScript, Python, etc.)	Funcionalidad nativa de lenguajes como JavaScript (JSON.parse()) y Python (json.loads()).	- Más eficientes en muchos casos que otras tecnologías. - No requieren aprender otro lenguaje.	- Funcionan únicamente cuando trabajamos directamente sobre los archivos. - No tienen una sintaxis de consulta declarativa como *JSONPath* o *JQ*.
Lenguajes y librerías propias de bases de datos NoSQL	Normalmente, cada sistema de base de datos NoSQL tiene su propio lenguaje	- Nos permitirán interactuar con tecnologías de más alto rendimiento y capacidad.	- Curva de aprendizaje más lenta. - Requieren infraestructuras más complejas

En la unidad anterior, ya hemos visto tecnologías como ***JSONPath*** y ***JSONata***. No tiene mucho sentido que estudiemos con detalle todas las posibilidades que existen, por ello, se mostrarán a continuación una serie de ejemplos de las otras tecnologías, para que os hagáis una idea de cómo podemos trabajar con ellas. **Dejaremos para otro documento el trabajo sobre MongoDB, que es a día de hoy, el sistema de bases de datos NoSQL documentales más usado**.

5 Ejemplos de uso de algunas tecnologías

En los siguientes apartados veremos algunos ejemplos de distintas tecnologías que se pueden usar para consultar y procesar contenido JSON. Tomaremos como referencia el siguiente documento:

```
{
 "nombre": "Carlos",
 "edad": 40,
 "contacto": {
  "email": "carlos@example.com",
  "telefono": "555-1234"
 },
 "hobbies": ["fútbol", "cine", "cocina"],
 "activo": true,
 "proyectos": [
  {
   "titulo": "Desarrollo de Aplicación",
   "completado": false,
   "lenguajes": ["Python","Java","C#"]
  },
  {
   "titulo": "Diseño de Página Web",
   "completado": true,
   "lenguajes": ["Python","Java"]
  }
 ]
}
```

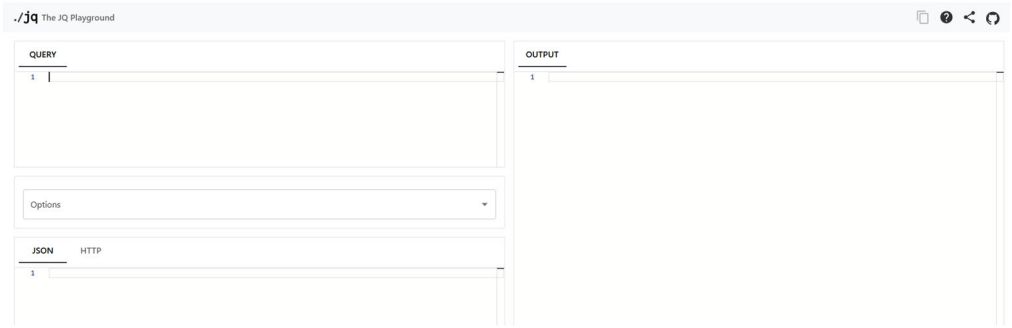
5.1 JQ

JQ es una poderosa herramienta de línea de comandos para procesar JSON. Funciona como un lenguaje funcional que permite **filtrar, transformar y manipular** datos JSON de manera eficiente. Se puede usar en terminales de cualquier sistema operativo: Linux, MacOS o Windows.

¿Cómo funciona JQ?

1. **Toma JSON como entrada** (puede ser un archivo o la entrada estándar).
2. **Ejecuta una consulta o transformación** sobre el JSON.
3. **Devuelve el resultado** (ya sea JSON modificado o valores extraídos).

Podemos ver el resultado que obtendríamos con una expresión determinada de JQ en la siguiente web: https://play.jqlang.org/



Por ejemplo:

Obtener el nombre:

cat data.json | jq .nombre

Salida: "Carlos"

Obtener el correo electrónico:

cat data.json | jq .contacto.email

Salida: "carlos@example.com"

Listar elementos de un array:

cat data.json | jq .hobbies

Salida: ["fútbol", "cine", "cocina"]

Filtrar elementos de un array. Mostrar títulos de los proyectos no completados:

cat data.json | jq .proyectos[] | select(.completado == false) | .titulo

Salida: "Desarrollo de Aplicación"

Mostrar solo los títulos de los proyectos en un array:

cat data.json | jq '[.proyectos[].titulo]'

Salida: ["Desarrollo de Aplicación", "Diseño de Página Web"]

Crear un nuevo JSON con solo nombre y proyectos:

cat data.json | jq '{nombre: .nombre, proyectos: .proyectos}'

Salida:

OUTPUT


```
{
  "nombre": "Carlos",
  "proyectos": [
    {
      "titulo": "Desarrollo de Aplicación",
      "completado": false,
      "lenguajes": [
        "Python",
        "Java",
        "C#"
      ]
    },
    {
      "titulo": "Diseño de Página Web",
      "completado": true,
      "lenguajes": [
        "Python",
        "Java"
      ]
    }
  ]
}
```


Obtener una lista de todos los lenguajes usados en proyectos:

cat data.json | jq ' [.proyectos[].lenguajes[]] | unique'

Salida: ["C#", "Java", "Python"]

¿Cuántos lenguajes distintos se usan?

cat data.json | jq '[.proyectos[].lenguajes[]] | unique | length'

Salida: 3

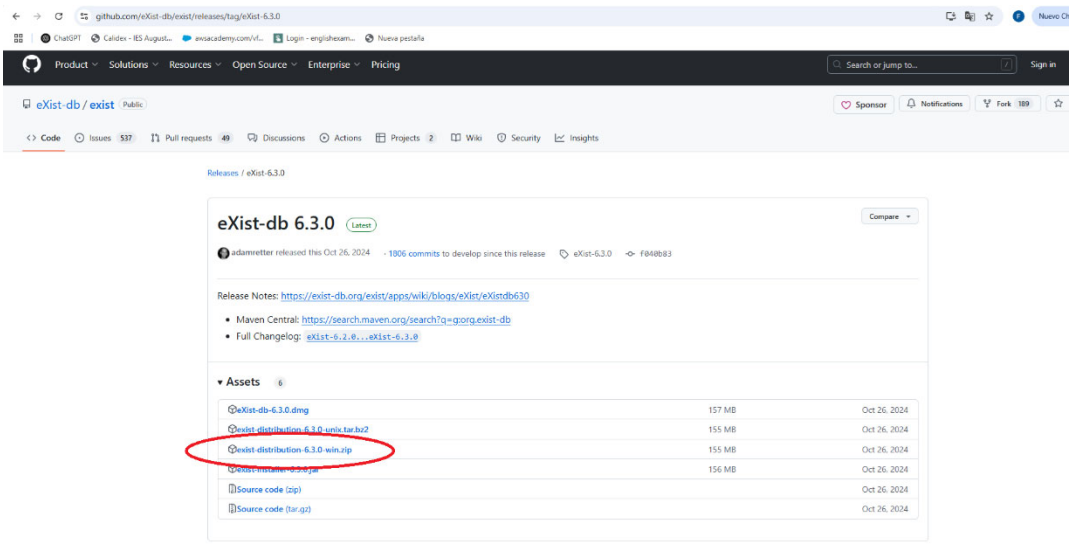
5.2 JSONiq

JSONiq es un **lenguaje de consulta y transformación de JSON** inspirado en **XQuery** (usado para XML). A diferencia de **JQ**, que es una herramienta de línea de comandos, JSONiq está más orientado a bases de datos JSON, motores de almacenamiento y procesamiento avanzado.

JSONiq está implementado en varias herramientas. Por ejemplo, en este documento vamos a usar la herramienta **eXist-db**, que permite crear colecciones de ficheros XML y JSON y consultarlos posteriormente con *XQuery* y *JSONiq:*

https://exist-db.org/exist/apps/homepage/index.html)

Esta herramienta trabaja sobre Java, por lo que necesitamos tener una máquina de Java ejecutándose en nuestro equipo. La instalación es muy sencilla. En entornos Windows, basta con descargarse el fichero .zip y descomprimirlo en la ubicación que queramos:

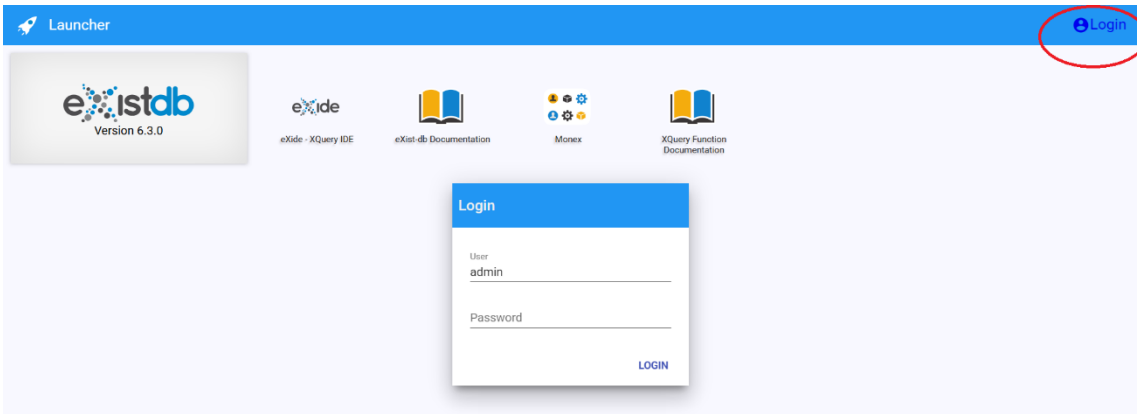


Para poner en marcha el servidor, debemos irnos a la carpeta ***bin*** y ejecutar el fichero ***startup.bat***

La herramienta dispone de una interface de usuario de tipo web, a la que podemos acceder a través de la siguiente URL:

http://localhost:8080/exist

Al lanzarlo la primera vez, veremos algo como lo siguiente. Lo primero que tenemos que hacer es hacer ***login*** con el usuario con el que queramos trabajar.

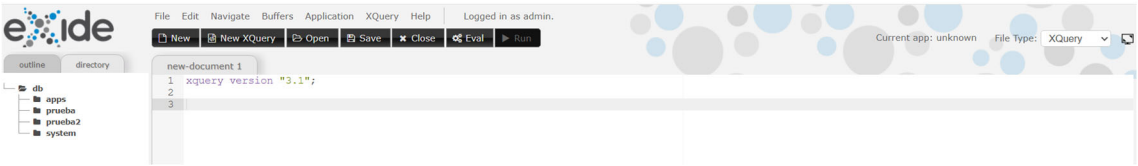


Por defecto, vienen varios usuarios definidos. El que tiene todos los privilegios es el usuario ***admin***, cuya contraseña inicial ***está vacía***.

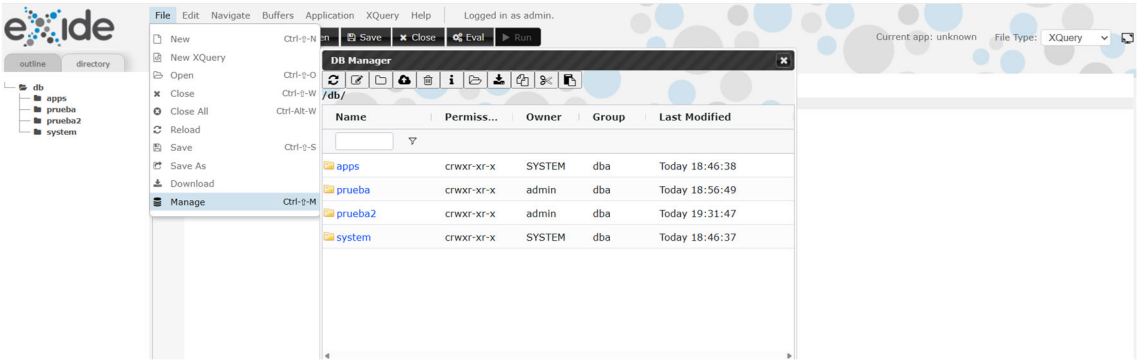
Una vez identificados, bastará con poner en marcha el ***XQuery IDE*** para poder interactuar con los datos:



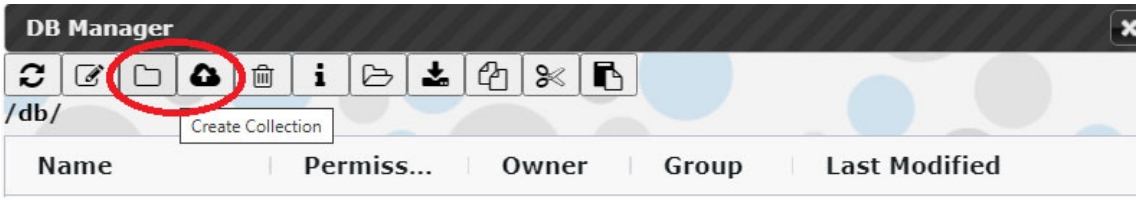
Veremos algo parecido a lo siguiente:



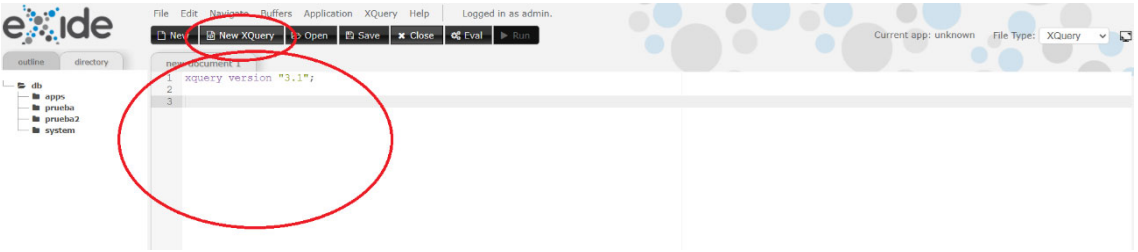
Los contenidos se organizan en lo que se llaman ***colecciones***. Las *colecciones* tienen una **estructura de carpetas** y las podemos administrar desde la opción del menú ***File → Manage***

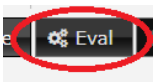


En nuestro caso, hemos creado dos colecciones con los nombres de ***prueba*** y ***prueba2*** y dentro hemos subido los ficheros JSON y XML con los que trabajar.



Ya estamos en disposición de comenzar a escribir nuestras queries y ver los resultados obtenidos. Para ello, haremos click sobre la opción ***New XQuery*** y en el cuadro de edición que nos sale, podremos escribir nuestras consultas.



Para ejecutar la consulta, pincharemos sobre la opción ***Eval:*** 

La forma básica de referenciar un documento JSON es la siguiente:

```
xquery version "3.1";
let $json := json-doc("/db/prueba/datos.json")
```

Por su parte, si estuviéramos consultando un documento XML con utilizaríamos la función ***doc***:

```
xquery version "3.1";
let $json := doc("/db/prueba2/datos.xml")
```

Veamos a continuación, cómo hacer los mismos ejemplos que vimos con ***JQ***, con ***JSONiq***:

Obtener el nombre:

```
xquery version "3.1";
let $json := json-doc("/db/prueba2/datos.json")
return $json("nombre")
```

Salida: "Carlos"

Obtener el correo electrónico:

```
xquery version "3.1";
let $json := json-doc("/db/prueba/datos.json")
return $json("contacto")("email")
```

Salida: "carlos@example.com"

Listar elementos de un array:

```
xquery version "3.1";
let $json := json-doc("/db/prueba/datos.json")
return $json("hobbies")
```

Salida: ["fútbol", "cine", "cocina"]

Filtrar elementos de un array. Mostrar títulos de los proyectos no completados:

```
xquery version "3.1";
let $docum := json-doc("/db/prueba/datos.json")
let $proyectos := $docum("proyectos")
let $proyectosFiltrados := array:filter($proyectos, function($p) {
  $p("completado") = true()
})
return array:for-each($proyectosFiltrados, function($p) {
  $p("titulo")
})
```

Salida: [“Desarrollo de Aplicación"]

Crear un nuevo JSON con solo nombre y proyectos:

```
xquery version "3.1";
let $doc := json-doc("/db/prueba/datos.json")
return map {
  "nombre" : $doc("nombre"),
  "proyectos" : $doc("proyectos")
}
```

Salida:



Mostrar solo los títulos de los proyectos en un array:

```
xquery version "3.1";
let $docum := json-doc("/db/prueba/datos.json")
let $proyectos := $docum("proyectos")
return array:for-each($proyectos, function($p) {
  $p("titulo")
})
```

Salida: ["Desarrollo de Aplicación", "Diseño de Página Web"]

Obtener una lista de todos los lenguajes usados en proyectos:

```
xquery version "3.1";
let $doc := json-doc("/db/prueba/datos.json")
let $res :=  array:for-each($doc("proyectos"), function($p) {
  $p("lenguajes")
})  (:  $res contiene un array de arrays :)

let $distinct := distinct-values($res) (: Quitamos repetidos :)
return $distinct
```

Salida: "C#", "Java", "Python"

¿Cuántos lenguajes distintos se usan?

```
xquery version "3.1";
let $doc := json-doc("/db/prueba/datos.json")
let $res :=  array:for-each($doc("proyectos"), function($p) {
  $p("lenguajes")  })  (:  $res contiene un array de arrays :)
let $distinct := distinct-values($res) (: Quitamos repetidos :)
return count($distinct)
```

Salida: 3